

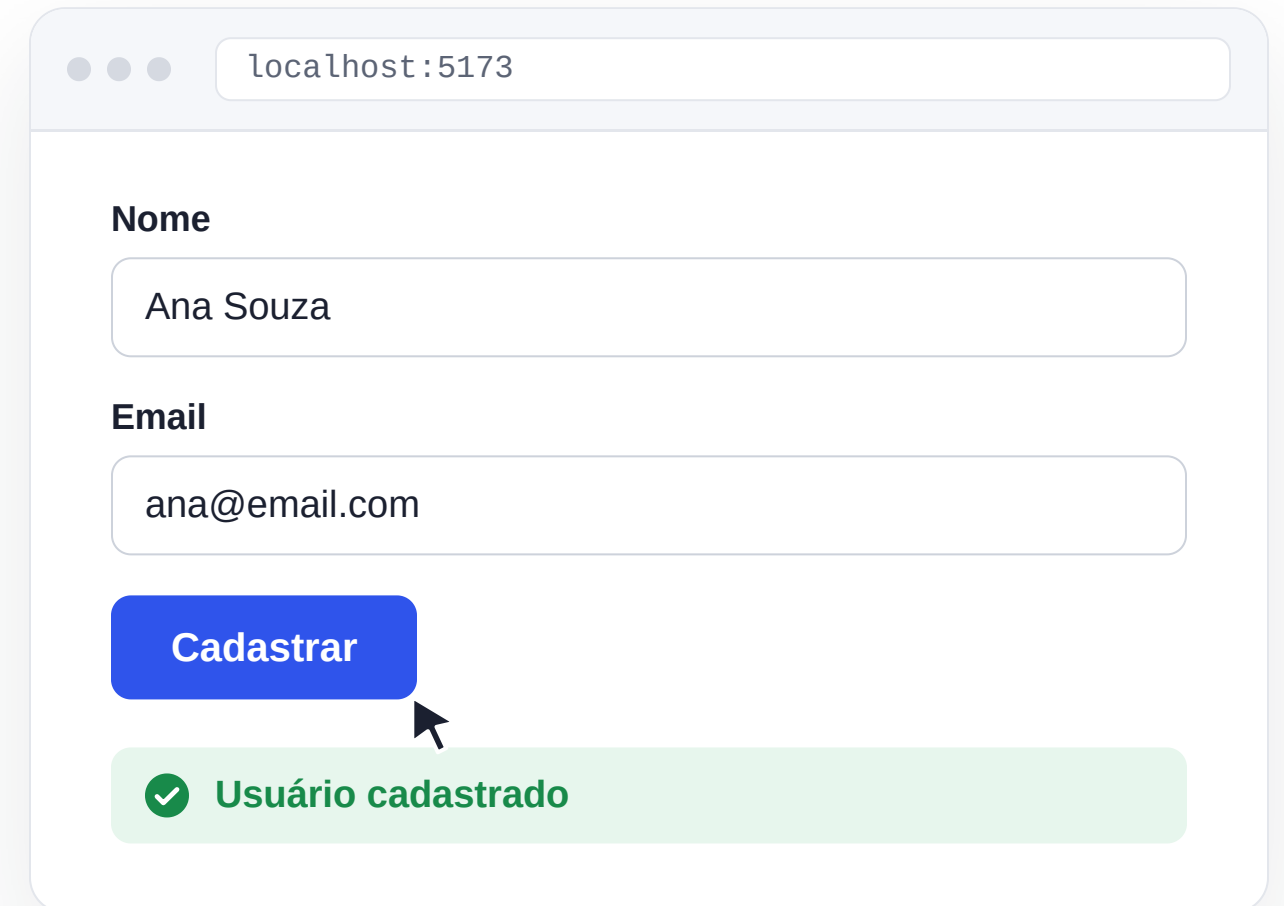
Testes de Software

Testes E2E

Testando o sistema inteiro do jeito que o usuário usa: abrindo o navegador, clicando e lendo a tela.

Professores

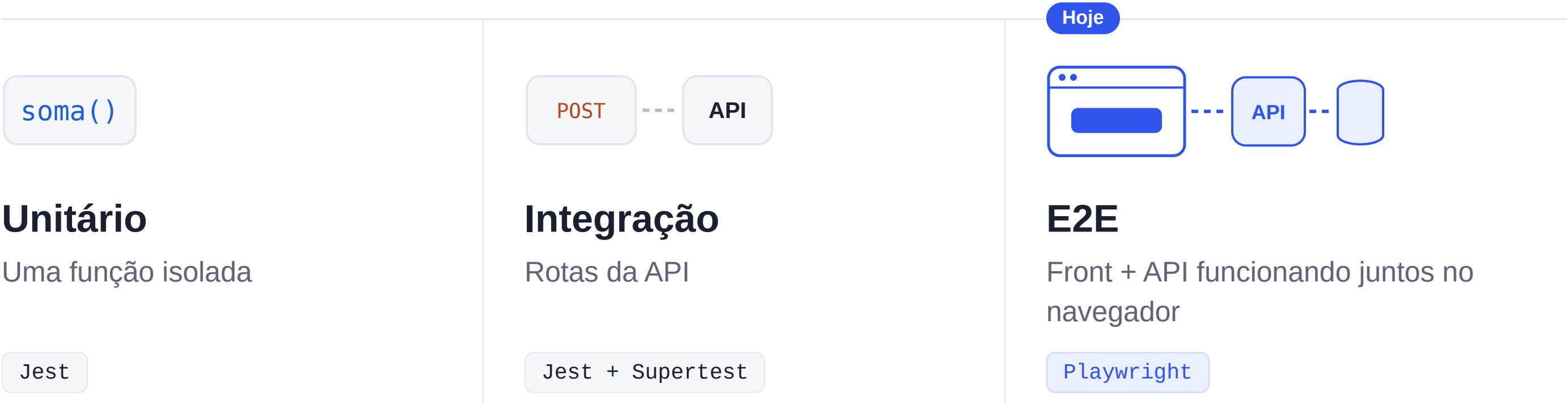
Celso Rodrigo Giusti (The Best)
Felipe Augusto Stevanim Gregate
Daniel Manoel Filho
Marlon Palata Fanger Rodrigues



A screenshot of a web browser window with the address bar showing 'localhost:5173'. The page contains a registration form with two input fields: 'Nome' (Name) with the value 'Ana Souza' and 'Email' with the value 'ana@email.com'. Below the email field is a blue button labeled 'Cadastrar'. At the bottom of the form, there is a green success message: '✓ Usuário cadastrado'. A mouse cursor is pointing at the 'Cadastrar' button.

Relembrando

Já testamos funções e rotas da API. Falta testar o **sistema inteiro**, do jeito que o usuário usa.



O que é

Teste E2E

End to End, "de ponta a ponta": testa um fluxo completo, da tela até o banco de dados, simulando um usuário de verdade.



Pega erros que os outros testes não pegam. A função passa, a rota passa... mas o botão da tela não chama a API. Só o E2E percebe.

O que é

Pirâmide de testes



Quanto mais alto, mais caro

E2E abre navegador, sobe front e API. É mais lento para rodar e mais trabalhoso de manter quando a tela muda.

Por isso, só os fluxos importantes

O caminho que não pode quebrar de jeito nenhum.

Cadastro

Login

Finalizar compra

O que é

Playwright

Ferramenta da Microsoft, gratuita e open source, que **controla navegadores por código** para automatizar testes.

Vários navegadores

Chromium
Chrome e Edge

Firefox
Mozilla

WebKit
Safari

Várias linguagens

Vamos usar **JavaScript/TypeScript**. Também existe para Python, Java e .NET.

Espera sozinho (auto-wait)

Aguarda o elemento aparecer antes de agir. Nada de

~~`setTimeout(..., 2000)`~~

Ferramentas prontas

Modo UI

Gravador de testes (codegen)

Relatório HTML

Anatomia do teste

O mesmo **AAA** que vimos no Jest:

Arrange

preparar

Act

agir

Assert

verificar

```
import { test, expect } from '@playwright/test';

test.beforeEach(async ({ page }) => {
  await page.goto('http://localhost:5173');
});

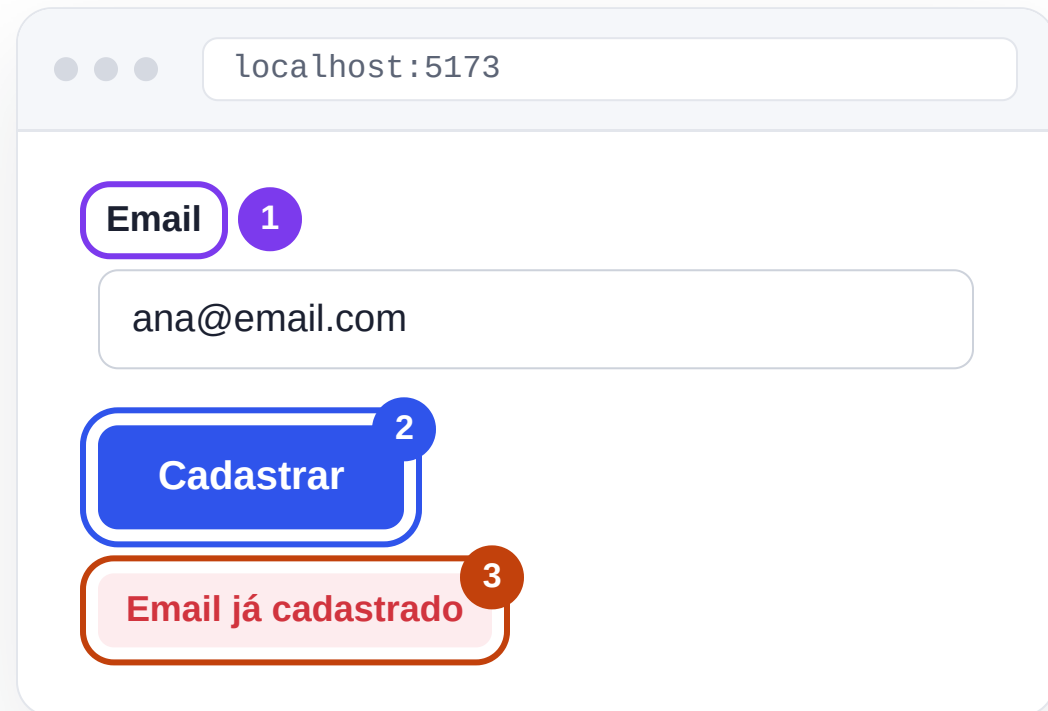
test('cadastra um novo usuário', async ({ page }) => {
  await page.getByLabel('Nome').fill('Ana');
  await page.getByLabel('Email').fill('ana@email.com');
  await page.getByRole('button', { name: 'Cadastrar' }).click();
  await expect(page.getByText('ana@email.com')).toBeVisible();
});
```

`page` representa a aba do navegador.

Toda ação usa `await`, porque o navegador responde de forma assíncrona.

`beforeEach` roda antes de cada teste: é a preparação comum.

Localizadores: como achar os elementos



Encontre os elementos do jeito que o usuário **enxerga a tela**: pelo rótulo, pelo papel ou pelo texto.

1 `page.getByLabel('Email')`

O campo, pelo rótulo

2 `page.getByRole('button', { name: 'Cadastrar' })`

Pelo papel do elemento e pelo texto dele

3 `page.getByText('Email já cadastrado')`

Qualquer texto da tela

Evite: classe CSS muda o tempo todo

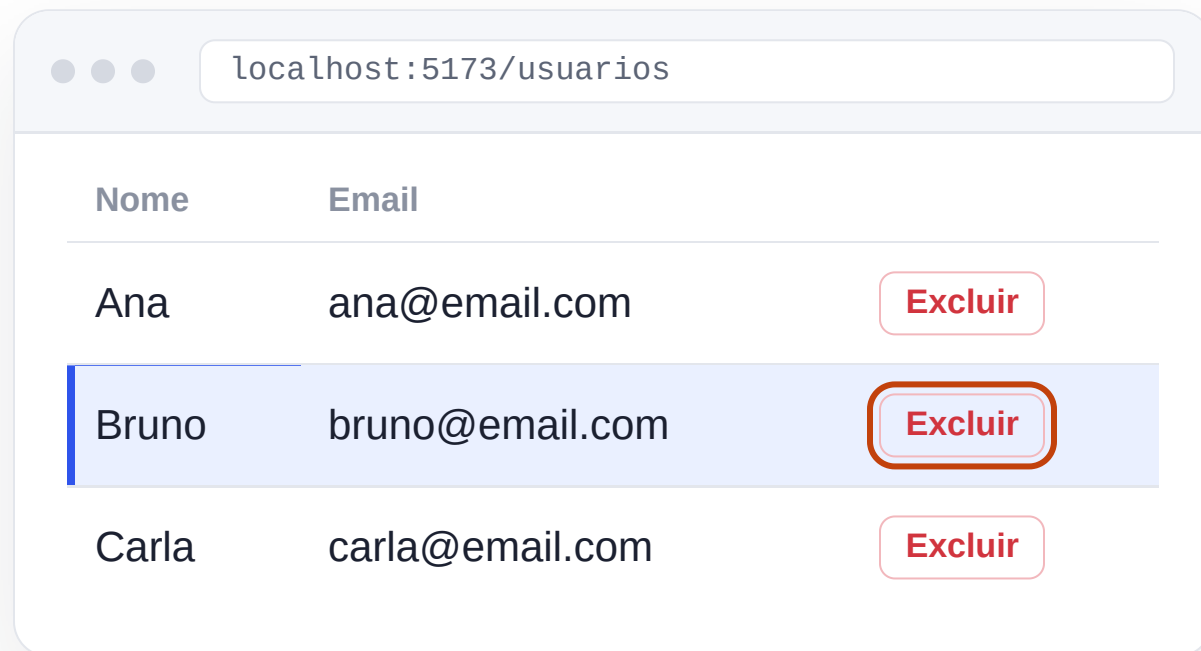
```
page.locator('.btn-primary.css-1x7fk')
```

Prefira: o que o usuário vê

```
page.getByRole('button', { name: 'Cadastrar' })
```

Encadeando localizadores

Quando existem vários botões iguais, primeiro achamos a **linha** e depois o **botão dentro dela**.



Nome	Email	
Ana	ana@email.com	Excluir
Bruno	bruno@email.com	Excluir
Carla	carla@email.com	Excluir

page

```
.getByRole('row', { name: 'Bruno' })  
.getByRole('button', { name: 'Excluir' })  
.click();
```

- 1 Acha a **linha** que contém "Bruno"
- 2 Dentro dela, acha o botão **Excluir**
- 3 Clica só nesse botão

Ações e verificações

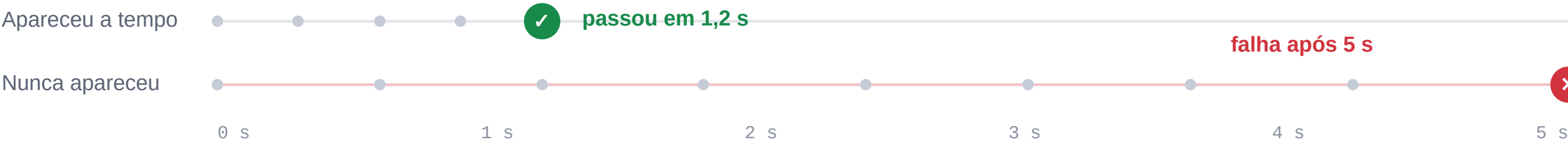
Ações simulam o usuário

<code>goto()</code>	abre uma página
<code>fill()</code>	digita num campo
<code>click()</code>	clica
<code>selectOption()</code>	escolhe num select

Verificações `expect()`

<code>toBeVisible()</code>	está na tela
<code>toContainText()</code>	contém o texto
<code>toHaveLength()</code>	tem N elementos
<code>toHaveValue()</code>	o campo tem o valor

Auto-wait: o expect tenta de novo até dar certo, por até 5 segundos



Atualizações no repositório

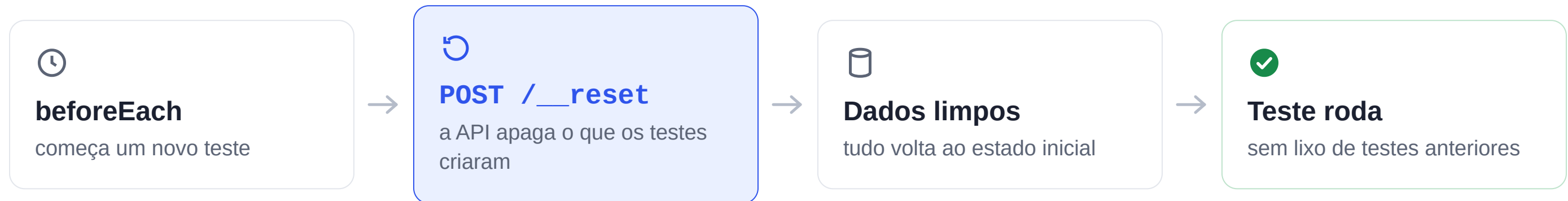
No mesmo repositório das aulas anteriores foi adicionada a pasta `web`: um front-end simples em **React**, onde vamos escrever os testes.

- 1 Sincronize o seu fork
No GitHub, botão **Sync fork**
- 2 Puxe os arquivos novos
`git pull`

github.com/dannmf/unit-tests-starter

```
unit-tests-starter/  
├─ arquivos das aulas anteriores  
└─ web/ nova  
    ├─ src/  
    └─ package.json
```

A rota /__reset



↺ e repete a cada teste

Nova rota **POST** na API. Sem ela, cada execução deixaria usuários cadastrados e um teste passaria a **depender do outro**.

```
test.beforeEach(async ({ page, request }) => {  
  await request.post('http://localhost:3000/__reset');  
  await page.goto('http://localhost:5173');  
});
```

Configuração do ambiente

Rodar dentro da pasta /web

```
terminal: unit-tests-starter/web

# instala o Playwright
$ npm install -D @playwright/test

# instala o navegador do Playwright
$ npx playwright install chromium

# grava um teste clicando na aplicação
$ npx playwright codegen http://localhost:5173
```

Scripts no package.json

```
"scripts": {
  "test:e2e": "playwright test",
  "test:e2e:ui": "playwright test --ui",
  "test:e2e:report": "playwright show-report"
}
```

⚠ O **codegen** só funciona com a aplicação rodando em `localhost:5173`.